Heat Conduction Simulation and Physics Informed Neural Networks (PINN)

Robert Haas

Documentation • September 29, 2025 • Version 1.0

PDF generated with LaTeX

Project page: https://github.com/Haasrobertgmxnet/HeatConduction
Contact: Haasrobert@gmx.net

Abstract

This documentation presents numerical studies for the 2D heat conduction equation. The heat equation is solved using classical numerical methods, such as explicit Euler and implicit Crank-Nicolson, as well as Physics Informed Neural Networks (PINNs). Comparisons between these methods are provided, highlighting stability, accuracy, and computational efficiency.

Contents

1 The classical Heat Equation

We now derive the heat equation from the energy balance. For further reading see [1, pp 207] and [2, pp 105] Heat conduction describes the transfer of thermal energy in materials and is governed by the heat equation. The mechanism of heat conduction takes places on the scale of atoms and molecules and their local motion as mechanism to transfer heat. We consider an open spatial domain $\Omega \subset \mathbb{R}^d$ where the heat conduction is investigated. Let $R \subset \Omega$ be an arbitrary subdomain of Ω with the following assumptions:

1. **Regularity:** R is regular, so that all following integrals and operations on R are well-defined.
2. **Absence of particle motion:** R does not move over time, i.e. macroscopic motion of particles inside R is not allowed.

The heat equation can be derived from energy balance as follows. The change in time of the internal energy $\mathscr{E}(R)$ is given by the heat flux crossing the boundary of R and the heat sources and sinks in R, i.e.

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathscr{E}(R) = \frac{\mathrm{d}}{\mathrm{d}t}\int_R \varrho u \mathrm{d}V = \int_R \varrho f \mathrm{d}V - \int_{\partial R} \boldsymbol{q}\cdot\boldsymbol{\nu}_{\partial R}\mathrm{d}A.$$

The used symbols have the following meaning:

Symbol	SI Unit	Description
ϱ	$\frac{\mathrm{kg}}{\mathrm{m}^d}$	Mass density
u	$\frac{\mathrm{J}}{\mathrm{kg}}$	Specific internal energy
f	$\frac{\mathrm{W}}{\mathrm{kg}}$	Specific hat sources
$\boldsymbol{q}$	$\frac{\mathrm{W}}{\mathrm{m}^{d-1}}$	Heat flux per area
$\boldsymbol{\nu}_{\partial R}$	1	Outer unit normal at the boundary of R

Table 1: Physical quantities.

Since R is regular we can interchange time derivation and integration and apply the Gauss-Green theorem on the boundary integral to obtain

$$\int_R \frac{\partial}{\partial t}(\varrho u)\,\mathrm{d}V = \int_R (\varrho f - \mathrm{div}(\boldsymbol{q}))\,\mathrm{d}V, \text{ or}$$

$$\frac{1}{\mathrm{meas}(R)}\int_R \frac{\partial}{\partial t}(\varrho u)\,\mathrm{d}V = \frac{1}{\mathrm{meas}(R)}\int_R (\varrho f - \mathrm{div}(\boldsymbol{q}))\,\mathrm{d}V.$$

Since R is an arbitrary sample volume, we can drop the integration to get

$$\frac{\partial}{\partial t}(\varrho u) = \varrho f - \mathrm{div}(\boldsymbol{q}).$$

We now fix the following assumptions:

(A1) Heat conduction is the only mechanism of heat transfer. That means convection and radiation are not present.

(A2) Heat sources and sinks in the bulk are nor present.

(A3) The specific internal energy is given by $u = c_P \cdot T$, where c_P is the specific heat capacity at constant pressure and T is the temperature.

(A4) The heat flux per area is given by $\boldsymbol{q} = -\kappa \nabla T$ (Fourier's law), where κ denotes the thermal conductivity.

(A5) The mass density ϱ and the specific heat capacity c_P do not change over time.

(A6) The thermal conductivity κ is a constant in space.

These assumptions lead to the classical heat conduction equation given by

$$\varrho c_P \frac{\partial T}{\partial t} = \kappa \Delta T, \text{ or} \tag{1}$$

$$\frac{\partial T}{\partial t} = \alpha \Delta T \text{ in } \Omega, \tag{2}$$

with the thermal diffusivity $\alpha = \kappa / c_P / \varrho$. The following table assigns the physical units to the quantities just introduced.

Symbol	SI Unit	Description	Typical Value
c_p	$\frac{\mathrm{J}}{\mathrm{kg \cdot K}}$	Heat capacity at constant pressure	4181
κ	$\frac{\mathrm{W}}{\mathrm{m \cdot K}}$	Heat conductivity	0.6
α	$\frac{\mathrm{m}^{d-1}}{\mathrm{s}}$	Thermal diffusivity	1.438 e-7

Table 2: Physical quantities.

General Assumption: From now on we assume that Ω is a 2D quadratic domain $\Omega = (0.1) \times (0, 1)$. Then the classical heat equation can be written as

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right) + f = \alpha \Delta T + f. \tag{3}$$

2 Finite Difference Method

The ideas of finite difference method is to approximate spatial derivatives by numerical difference quotients. Evaluating a difference quotients at a certain point (x, y) requires its neighboring points $(x - \Delta x, y)$, $(x + \Delta x, y)$, $(x, y - \Delta y)$ and $(x, y + \Delta y)$. For this purpose a set of grid points $\{(x_i, y_j) \| (i, j) \in I\}$ in Ω. Since $\Omega = (0, 1) \times (0, 1)$ the grid points $\{(x_i, y_j)$ are aligned with the coordinate axes. Setting $h = \Delta x = \Delta y$ we approximate the second order spatial derivatives by

$$\begin{aligned} \Delta v(x, y) &\approx \Delta_h v(x, y) \\ &= \frac{1}{h^2} \left(v(x + h, y) - 2v(x, y) + v(x - h, y) \right) \\ &+ \frac{1}{h^2} \left(v(x, y + h) - 2v(x, y) + v(x, y - h) \right), \end{aligned}$$

and the first order time derivative by a forward difference

$$\frac{\mathrm{d}w(t))}{\mathrm{d}t} \approx \frac{w(t+\tau)}{\tau}.$$

We now use u as an alias for T, i.e. $u = T$. With a definite (x, y) grid and an initial state $T_0(x, y)$ at time $t = 0$ an approximation for the solution u at time $t = 0 + \tau$ can be obtained via

$$u_1 = u_0 + \alpha h \Delta_h u_0,$$

where $u_0 = T_0$ evaluated at grid points. This can be successive be done:

$$u_{k+1} = F_{\text{explicit}}(u_k) = u_k + \alpha h \Delta_h u_k,$$

what defines the explicit Euler method. In contrast the implicit method is given via

$$u_{k+1} = F_{\text{implicit}}(u_k, u_{k+1}) = u_k + \alpha h \Delta_h u_{k+1},$$

which requires to solve a system of linear equations to get u_{k+1}, but the numerical behavior of the implicit method is more robust. Combining explicit and implicit method leads to the Crank-Nicolson method as arithmetic mean of both methods

$$u_{k+1} = \tfrac{1}{2}\left(F_{\text{explicit}}(u_k) + F_{\text{implicit}}(u_k, u_{k+1})\right).$$

The stability of the explicit Euler method is limited by the CFL condition:

$$\tfrac{\tau}{h^2} \leq \tfrac{1}{4\alpha},$$

whereas for the Crank-Nicolson method

$$\tfrac{\tau}{h^2} \leq \tfrac{1}{2\alpha}$$

must be fulfilled for L^∞-stability. For more information, see [3, p 112].

3 Physics Informed Neural Networks (PINN)

Classification or *approximation* are the typical applications of neural networks. A short introduction to neural networks can be found in [4, pp. 809], [5] and [6]. Today the main applications of neural networks contain classification. On the other hand, neural networks can be used to approximate an (unknown) function. Physics informed neural networks are a special type of neural networks to solve mathematical equations of physics and engineering. Typically these equations are ordinary or partial differential equations. Whereas classical numerical solvers outperform this deep learning approach, trained PINN models can be easily reused, if, e.g. a finer spatial grid is needed or the geometry of the spatial domain varies slightly. In good cases these tasks can be done without a new training. Furthermore PINNs can be used to estimate parameters in the equation, if an approximate solution is given, e.g. by measurements. In PINNs the neural network is used as a function approximator. This approach can be found in [7, Section 3.2]. The activation functions of a neural network span a function space and

a function is approximated by a linear combinations of copies of the activation functions each having different scaling and offset in its function argument. The coefficients, the scalings and offsets are simply weights of the neural network to be adjusted during training. Key ingredient of the learning procedure is the loss function which will be minimized during training. This loss function will be built up by loss terms arising from the mathematical equation to solve. That means that three loss terms will be added together to give the (total) training loss:

1. **Physics loss:** The loss arising from deviations of the approximate solution $\hat{u}$ from the exact solution u. Generally, if $F(u) = 0$ is the mathematical equation, then the physics loss is $L_{\text{Physics}}(w) = |F(w)|^2$ where w can be any admissible function. In case $w = u$ the equation $F(u) = 0$ is fulfilled and $L_{\text{Physics}}(u) = 0$ is minimal. For any approximate solution $\hat{u}$ we only have $L_{\text{Physics}}(\hat{u}) = 0$. So L_{Physics} penalizes deviations from the exact solution u.

2. **Initial loss:** Similarly, let $I(u) = 0$ denote the initial condition. Then the initial loss is defined as $L_{\text{Initial}}(w) = |I(w)|^2$.

3. **Boundary loss:** Finally, for a boundary operator $B(u) = 0$ the boundary loss is analogously defined as $L_{\text{Boundary}}(w) = |B(w)|^2$.

To set up the PINN training the approach of [8] is used and extended. The source code used therein can be found in [9]. In this thesis an inhomogeneous heat conduction equation in two spatial dimensions with Dirichlet or Neumann boundary conditions is solved using PINNs. Let us shortly summarize the key features:

1. **PINN architecture:** A feed forward neural network with five hidden layers, each of them containing 50 nodes is used. The network has input layer with three input nodes corresponding to the variables x, y, and t. The single node of the output layer is assigned to the approximate solution $\hat{u}$.

2. **PINN Activation function:** All layers except the output layer use the hyperbolic tangent as activation function.

3. **PINN Solver:** To train the PINN the Adam solver of the Pytorch package is used with default settings and a learning rate of 1e-3.

4. **Sampling:** Random samples of $N \in \{1000,\ 8000\}$ points, each for x, y and t, which is a Monte-Carlo sampling.

4 Case Studies

4.1 Case 1: Constant initial temperature with no-flux boundary conditions

This case is also discussed in [8]. With the introduces PINN setup, we use a thermal diffusivity $\alpha = 0.1$ in the heat equation. Furthermore we have a Gaussian kernel as a bulk heat source, i.e.

$$f(x, y, t) = S \cdot \exp\left(-\frac{(x^2+y^2)}{2r}\right),$$

with $r = 0.1$ and heat strength $S = 500.0$. As initial condition at $t = 0$ we set the temperature equal to 25 °C at every point in Ω. We assume that there is now heat flux across the boundary of Ω, i.e. $\frac{\partial u}{\partial n} = 0$ at $\partial\Omega$. For the training a Pytorch scheduler is used to adjust the learning rate adaptively. The adjustment starts after a certain number of finished epochs. This number is the step size parameter of the scheduler. We have used 4,000 and 6,000 as step size. After passing this number of epochs the learning rate decreases from 1e-3 to 5e-4 and further, each time by halving. Training with three models on the this non-homogeneous initial-boundary problem leads to the following Adjustments and results: The relative

Parameter	**Model 1**	**Model 2**	**Model 3**
Epochs	8,000	10,000	20,000
Samples	1,000	8,000	8,000
Step size	4,000	6,000	6,000
Relative error	19.03%	7.41 %	4.96 %

Table 3: Adjustment and Results.

error refers to the corresponding solution using the explicit finite-difference scheme. With more training epochs the relative error decreases.

Spatial Average of the calculated Temperatures

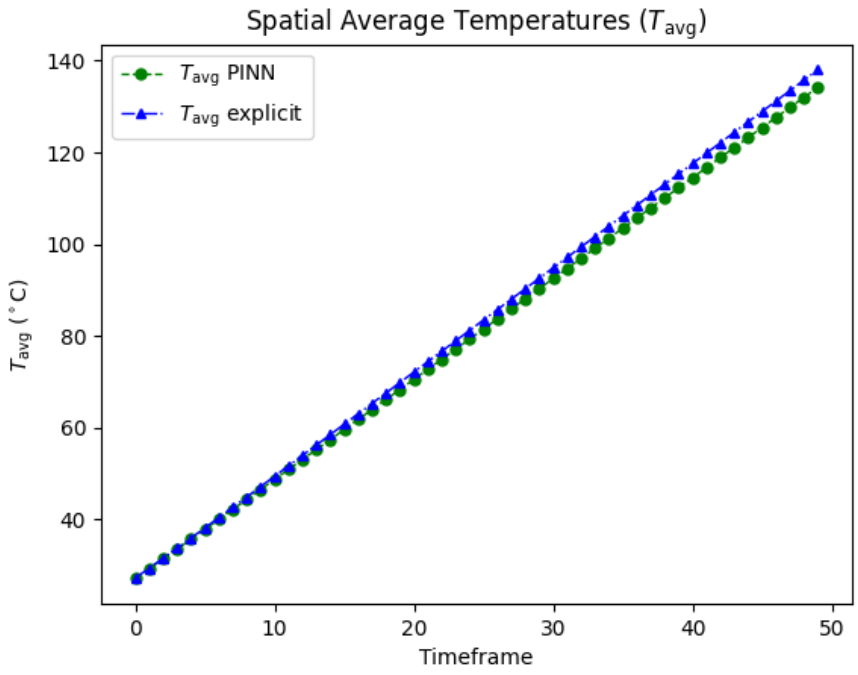


Table 4: Temperature averages for explicit and PINN solver.

The behaviour of the averaged temperatures shown in Figure (7) reveals a serious problem which is addressed by the following theorem.

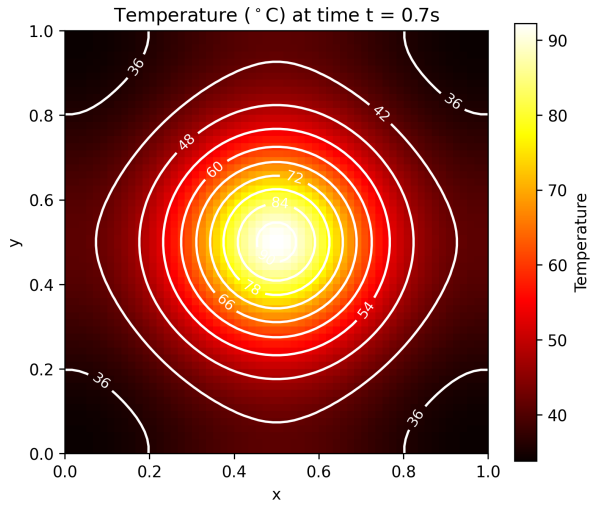
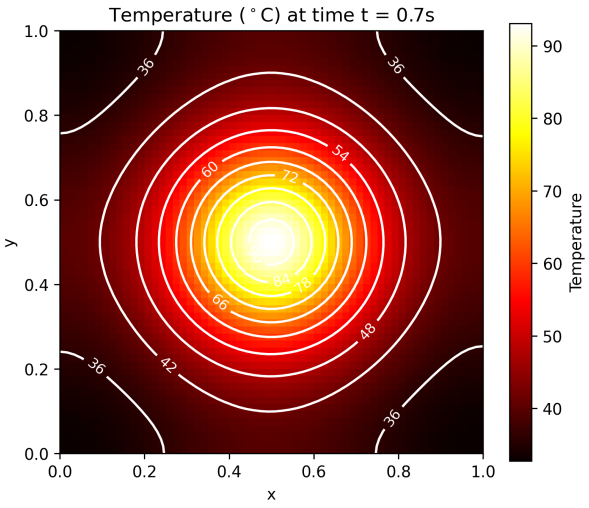
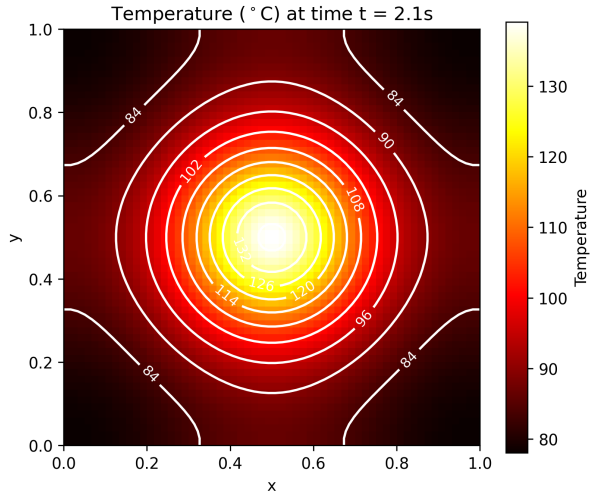
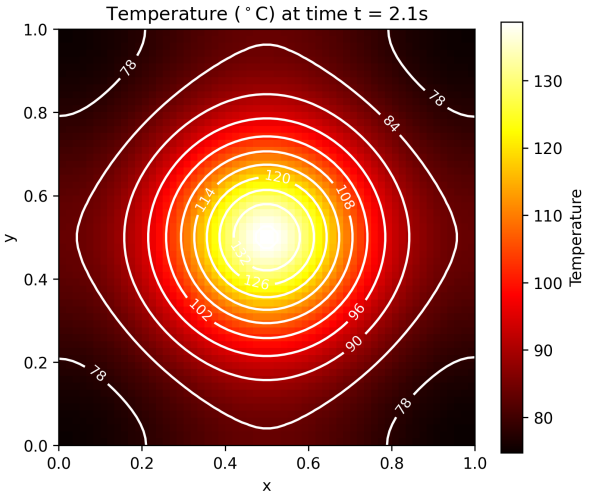
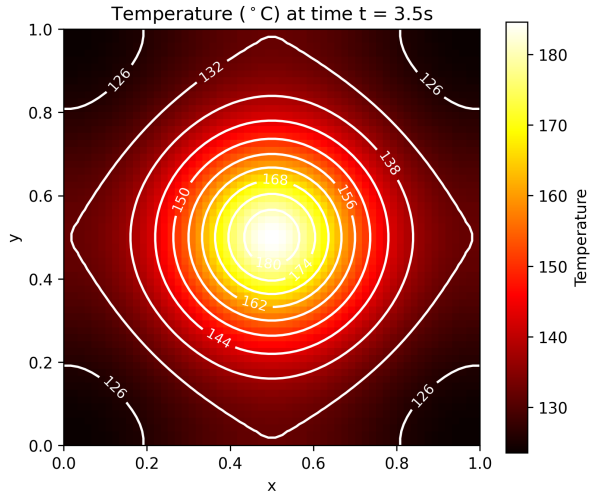
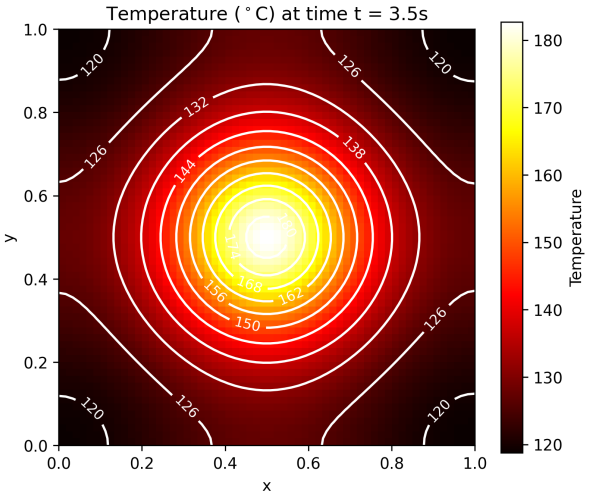
Explicit Solver	PINN
	
	
	

Table 5: Solution to different time frames.

Theorem 4.1 (Long-term behavior of the solution)**.** *With no-flux boundary conditions, i.e. zero normal derivatives of u at the boundary of Ω and time-constant bulk heating by internal heat sources f solutions u of the heat equation (3) will attain inifinite large temperatures when the time t tends to zero.*

Proof. We consider the time-space integrated version of (3), i.e.

$$\int_0^t \int_\Omega \frac{\partial u}{\partial t} \mathrm{d}\mu \mathrm{d}t = \alpha \int_0^t \int_\Omega \Delta u \mathrm{d}\mu \mathrm{d}t + \int_0^t \int_\Omega f \mathrm{d}\mu \mathrm{d}t.$$

Modifying the left hand side we have

$$\begin{aligned} \int_0^t \int_\Omega \frac{\partial u}{\partial t} \mathrm{d}\mu \mathrm{d}t &= \int_\Omega \int_0^t \frac{\partial u}{\partial t} \mathrm{d}t \mathrm{d}\mu \\ &= \int_\Omega u(t) \mathrm{d}\mu - \int_\Omega u(0) \mathrm{d}\mu. \end{aligned}$$

Since f does not depend upon time t we obtain

$$\begin{aligned} \int_\Omega u(t) \mathrm{d}\mu &= \int_\Omega u(0) \mathrm{d}\mu + \alpha \int_0^t \int_\Omega \Delta u \mathrm{d}\mu \mathrm{d}t + \int_0^t \int_\Omega f \mathrm{d}\mu \mathrm{d}t \\ &= \int_\Omega u(0) \mathrm{d}\mu + \alpha \int_0^t \int_{\partial\Omega} \frac{\partial u}{\partial \nu_{\partial\Omega}} \mathrm{d}S \mathrm{d}t + t \int_\Omega f \mathrm{d}\mu, \end{aligned}$$

where we have applied the Gauss-Green theorem, see [10, p. 288] and [11, p.312], on $\int_0^t \int_\Omega \Delta u \mathrm{d}\mu \mathrm{d}t$. Now, using the no-flux boundary condition we have

$$\begin{aligned} \sup_{(x,y)\in\Omega} |u(x,y)| &\geq \frac{1}{\mu(\Omega)} \int_\Omega u(t) \mathrm{d}\mu \\ &\geq \frac{\alpha}{\mu(\Omega)} \int_0^t \int_{\partial\Omega} \frac{\partial u}{\partial \nu_{\partial\Omega}} \mathrm{d}S \mathrm{d}t + \frac{t}{\mu(\Omega)} \int_\Omega f \mathrm{d}\mu \\ &\geq \frac{\alpha}{\mu(\Omega)} \int_0^t \int_{\partial\Omega} 0 \ \mathrm{d}S \mathrm{d}t + \mathrm{const} \cdot t \\ &\geq \mathrm{const} \cdot t, \end{aligned}$$

that is $\|u\|_{L^\infty(\Omega)} = \sup_{(x,y)\in\Omega} |u(x,y,t)|$ tends to infinity as $t \to \infty$. □

4.2 Case 2: Constant initial temperature with Robin boundary conditions

In case 1 the domain Ω is constantly heated over time where the heat cannot dissipate over across the boundary $\partial\Omega$ due to the no-flux boundary condition imposed on $\partial\Omega$. This leads to permanently ascending temperatures over time without reaching an equilibrium. In real life such an isolated system with permanent constant heat support is not realistic. This can be overcome by either

- using another heat source which decreases heat supply over time to zero, or
- imposing other boundary conditions which allow for heat flux over the domain boundary.

In this case the latter approach is used by imposing a Robin boundary condition, i.e.

$$a \cdot (u - u_0) + b \frac{\partial u}{\partial \nu_{\partial\Omega}} = 0,$$

where $\nu_{\partial\Omega}$ is the outer unit normal at the boundary $\partial\Omega$, u_0 the ambient temperature far from the domain Ω and its boundary $\partial\Omega$, i.e. far from the thermal boundary layer. With the choice $u_0 = 25\,^\circ\mathrm{C}$, $a = 1/2$ and $b = 1$ we redo the simulation of case 1 where all other parameters are left unchanged.

Parameter	**Model 1**	**Model 2**	**Model 3**
Epochs	8,000	10,000	20,000
Samples	1,000	8,000	8,000
Step size	4,000	6,000	6,000
Relative error	7.86%	3.92 %	3.73 %

Table 6: Adjustment and Results.

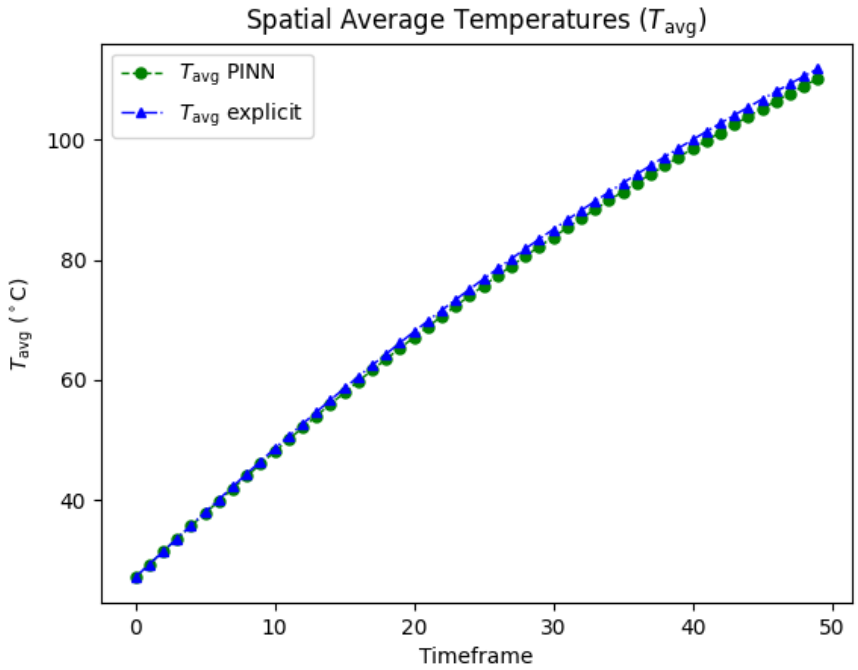
Spatial Average of the calculated Temperatures


Table 7: Temperature averages for explicit and PINN solver..

References

[1] C. Eck, H. Garcke, and P. Knabner, *Mathematische Modellierung* 1st. ed., Springer-Verlag Berlin Heidelberg, 2008.

[2] K.-H. Hoffmann and G. Witterstein, *Mathematische Modellierung: Grundprinzipien in Natur- und Ingenieurwissenschaften*, Mathematik Kompakt, 1st ed. (corrected reprint), Birkhäuser, Basel, 2014.

[3] O. Ernst, *Numerik partieller Differentialgleichungen, Part 2*, Lecture notes, University of Technology Chemnitz, 2015. Link (checked at 2025-09-10): https://www.tu-chemnitz.de/mathematik/numa/lehre/pde-2015/Folien/pdeTeil2.pdf

[4] H. Stöcker, *Taschenbuch mathematischer Formeln und moderner Verfahren*, 3rd rev. and ext. ed., Harri Deutsch Verlag, Frankfurt am Main, 1995.

[5] G. D. Rey, and K. F. Wender *Neuronale Netze*, 2nd rev. and ext. ed., Verlag Hans Huber, Bern, 2011

[6] R. Rojas, *Theorie der neuronalen Netze: Eine systematische Einführung*, Springer-Lehrbuch, 1st ed. (corrected reprint 1996), Springer-Verlag, Berlin, 1993.

[7] R. López González *Neural Networks for Variational Problems in Engineering* PhD thesis, Technical University of Catalonia, 2008 [Online]. Available: https://digital.csic.es/bitstream/10261/310317/1/Tesis_Roberto_LÃşpez.pdf

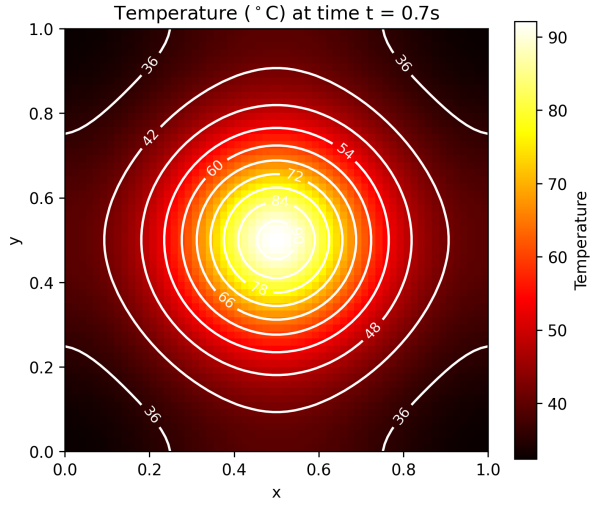
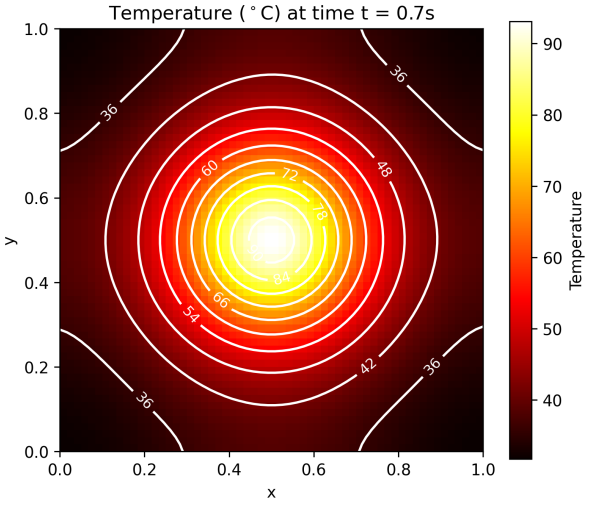
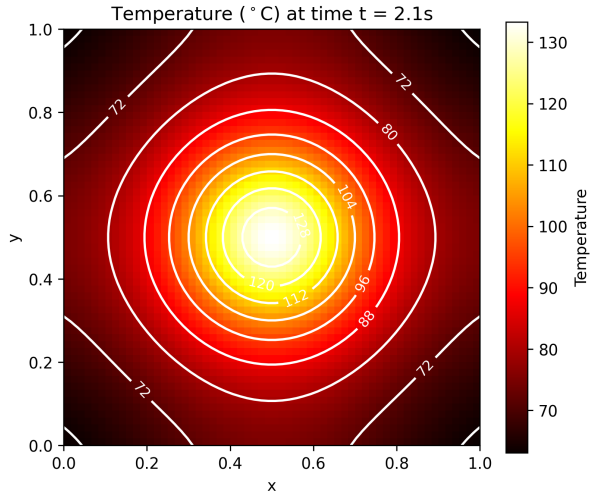
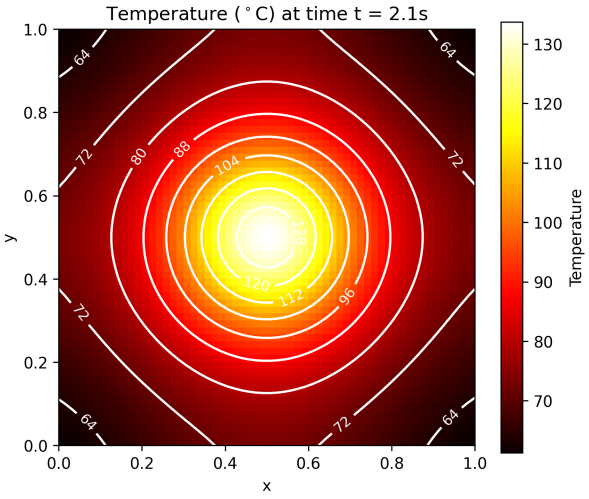
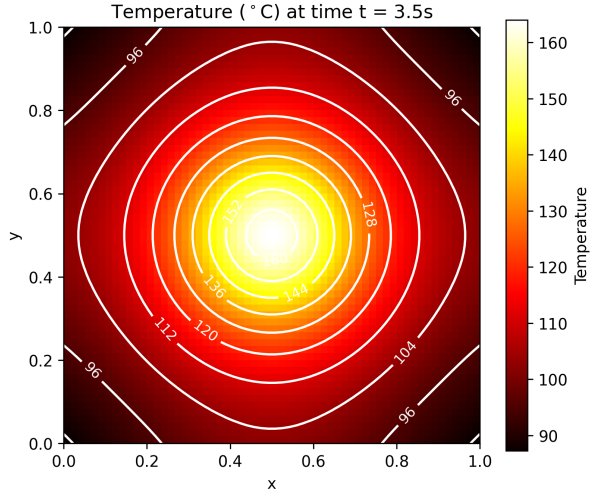
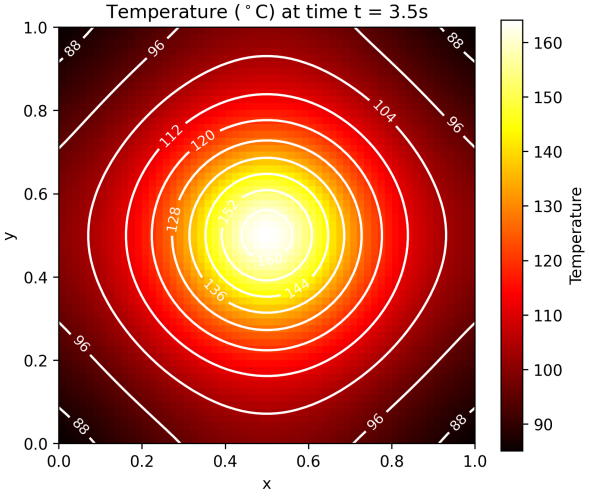
Explicit Solver	PINN
	
	
	

Table 8: Solution to difeerent time frames.

[8] S. Aryal, *Simulation of 2-dimensional heat conduction with physics-informed neural networks.* Bachelor thesis, University of Applied Sciences Ravensburg-Weingarten, 2024

[9] S. Aryal, *Bachelor thesis: Pinns for 2d heat conduction.* Sourcecode of [8], Link (checked at 2025-09-10): https://github.com/Samman2571/Bachelor_thesis_PINNs_2D_Heat_Conduction

[10] *Principles of Mathematical Analysis*, 3rd ed., McGraw-Hill International Editions

[11] *Mathematical Analysis*, Corrected 3rd printing., Springer-Verlag Berlin Heidelberg New York, 2001.

[12] Code samples used in this work: https://github.com/Haasrobertgmxnet/HeatConduction